# 计算机系统基础PA2-1实验报告

实验人：葛家韬（241220071）
授课教师：汪亮 苏丰

# 摘要

在 PA2-0 “汇编基础知识” 部分，我主要学习了从高级语言到机器指令的转化过程，理解了汇编语句与机器指令的对应关系，并通过 gdb 进行了调试实践。虽然这些内容在计算机组成课程中已有涉及，但本次实验进一步加深了我对汇编与机器码关系的理解，使我认识到同一类指令可能因操作数来源与长度不同而对应多个 opcode，也掌握了用 gdb 调试 NEMU 的方法，为后续实验奠定了基础。
在 PA2-1 “指令的解码与执行” 实验中，我完整地体验了从装载到执行、从分析到实现的流程。首先在 NEMU 中初始化内存与 CPU 状态，将可执行文件加载入内存后，CPU 不断执行“取指—解码—执行—写回—更新 EIP”的循环以模拟真实硬件。然后分析测试用例的反汇编结果，结合 i386 手册查阅缺失指令的格式和功能，利用 NEMU 提供的 OPERAND 数据结构、modrm 解析函数和操作数读写接口，分类实现不同类型的指令。对于双目运算类指令，优先使用 make_instr_impl_2op 宏简化代码，每实现一条指令即在 opcode_entry 数组中注册函数指针，通过不断调试与测试逐步完善指令集。实验过程中，我深刻理解了 NEMU 装载与执行程序的机制，掌握了操作码、ModR/M、SIB、立即数及位移量在指令解码中的作用，并体会到宏机制在代码复用与框架设计中的重要意义。整个实验使我对 CPU 的指令执行流程、计算机体系结构以及底层程序运行原理有了系统而深入的认识。

# PA2-0-汇编基础知识

## 课程知识点

- 从高级语言到机器指令
- 汇编语句与机器指令
- 尝试用gdb调试程序

## 心得体会

这一章的内容其实在上学期的计算机组成课程中也包括了很多，但还是有所收获的。让我明白了一件事，对于同一个指令，可能有很多个opcode与之对应，他们是对于不同类型的操作数（来自寄存器的，来自内存的，不同寻址方式），也有很多不同的操作数长度。同时学会了用gdb的方法调试nemu。这一部分的内容给后面的实验打下了基础。

# PA2-1指令的解码和执行

- 程序执行的宏观过程和模拟
- 单条指令的解码与NEMU实现

## 实验流程

本实验流程包括从装载到执行、从分析到实现的完整闭环：首先在 NEMU 中初始化内存与 CPU 寄存器，将可执行文件加载入内存后进入执行循环，CPU 不断“取指—解码—执行—写回—更新 EIP”以模拟真实硬件；同时，先分析测试用例的反汇编结果，结合 i386 手册查阅缺失指令的详细格式与语义，确定需要支持的操作码与操作数形式；在实现阶段，依托 NEMU 提供的 OPERAND 数据结构、modrm 解析函数及操作数读写封装，按操作数类型分类补齐指令，对双目运算类优先使用 make_instr_impl_2op 宏以减少重复代码、统一行为；每补齐一条指令就把对应执行函数注册到 opcode_entry 的相应位置，随后通过不断迭代的单步调试与回归测试校验效果，逐步扩充并打磨指令集直至通过所有要求的测试用例。

## 心得体会

在这个部分中，我深入了解了NEMU装载与执行程序的完整流程，明白了程序从可执行文件到被CPU模拟执行的每一个步骤，直观地体会到计算机运行原理的底层机制。同时，我认识到汇编指令的本质只是机器码的助记符，它与机器指令一一对应，方便人类理解和编写，而真正被执行的始终是二进制指令序列。通过学习指令解码的方式，我掌握了操作码、ModR/M、SIB以及立即数和位移量等字段在指令中的作用，理解了CPU如何根据这些信息确定操作数与执行行为。这一过程不仅加深了我对汇编语言与机器指令关系的认识，也让我对计算机体系结构和指令执行机制有了更系统、更深刻的理解。

## 建议和疑惑

强烈建议设计以指令族为单位的测试样例，比如这个test统一测试add的实现，下一个统一测试sub的实现，这样容易定位问题所在。这个宏的语法是非常厉害的，第一次真正看懂了C++的宏，非常开心。比较疑惑为什么模拟浮点数的运算不能直接调用C语言实现好的浮点数机制，是不是会让效率更高一些？

## 实验问题

### 使用hexdump命令查看测试用例的.img文件，所显示的.img文件的内容对应模拟内存的哪一个部分？指令在机器中表示的形式是什么？

img文件是nemu中“绿色”的文件，对应的是起始地址为0x0的内存地址，是模拟硬盘RAMDISK部分。指令是以二进制机器码形式存储的，在机器上是从0x30000位置开始的，这部分二进制的机器码可以通过反汇编得到汇编指令代码。

### 如果去掉instr_execute_2op()函数前面的static关键字会发生什么情况？为什么？

static关键字可以将函数的可见性限制在当前编译单元内，即函数只进行内部链接。当在函数定义前加上static时，不同的源文件可以各自定义同名的instr_execute_2op()函数而互不影响；但如果去掉static，这些函数就会变成全局可见。由于许多指令都定义了同名的instr_execute_2op()函数，链接器在链接阶段会检测到多个同名的全局函数定义，从而导致重复定义错误。

### 为什么test-float会fail？以后在写和浮点数相关的程序的时候要注意什么？

错误的根源是在这条fldl指令上：

```
    300a2:       dd 05 18 10 03 00       fldl   0x31018
```

源码中的load指令将double转换为float，再传入fpu，造成了精度丢失：

```
    fpu_load(f2u((float)temp.dval));
```

在实现浮点运算过程中应该注意保留double的精度。也可以考虑比较浮点数的时候不采用等于而采用小于一个很小的值。

# 参考资料及声明

- 课程群中ppt资料PA2-0 PA2-1(1) PA2-1(2)
- PA框架代码
- Xingkai Yu 实验代码 https://github.com/GeeeekExplorer/NJU-ICS
- 张逸凯（171840708）实验报告 https://gitee.com/whutglc/NJUCS-Course-Material-from-YikaiZhang

- 实验过程中使用了ChatGPT提供思路与指导，实验报告中的部分排版与表达参考了ChatGPT生成的内容。所有核心实验过程与结论均由本人亲自完成并负责。